

COMP64401 Module 5: Temporal Logics and LTL

Structured Study Notes for Exam Revision

Generated from the uploaded lecture transcripts and Week 9+10 slide deck

2026-05-16

Contents

Topic and scope	4
1. Why temporal logic?	4
1.1 Recap: previous logics were static	4
1.2 Ontological choices about time	4
1.3 Fluid vs rigid statements	4
2. LTL syntax	5
2.1 Formal definition of LTL formulae	5
2.2 Intuition for the temporal operators	5
2.3 Examples of LTL formulae	5
2.4 Non-examples	6
3. LTL semantics	6
3.1 LTL valuations	6
3.2 Satisfaction relation	6
3.3 Common mistake: “until” does not mean “then stops”	7
4. Syntactic sugar	7
4.1 Conjunction, truth, and falsity	7
4.2 Eventually and globally	8
5. Worked semantic examples	8
5.1 Basic valuation example	8
5.2 Until example 1	9
5.3 Until example 2	10
5.4 Until with bottom	10
6. Useful semantic consequences	10
6.1 If ψ holds now, then anything-until- ψ holds now	10
6.2 Eventually means “at some future point”	11
6.3 Two eventualities do not imply one combined eventuality	11
6.4 Important unfolding property of until	11
7. Transition systems: describing systems in LTL	12
7.1 Motivation	12
7.2 Formal definition of a transition system	12
7.3 Traces	12
7.4 Valuation associated with a trace	13
7.5 Key intuition	13
8. Worked transition-system example: car-driving abstraction	13

8.1 States and variables	13
8.2 Example traces	14
8.3 Associated valuations	14
8.4 Important abstraction point	15
9. LTL reasoning tasks	15
9.1 Satisfaction at a time point	15
9.2 Satisfaction by a valuation	16
9.3 Satisfiable in a valuation	16
9.4 Valid in a valuation	16
9.5 Satisfiable	16
9.6 Valid	16
10. Worked reasoning-task examples	16
10.1 Basic satisfaction	17
10.2 Until property	17
10.3 Global until property	17
10.4 Formula that is not valid in the valuation	17
11. Model checking	18
11.1 Formal definition	18
11.2 Universal vs existential model checking	18
11.3 Model-checking examples from the car system	18
12. Types of system properties	19
12.1 Local properties	19
12.2 Safety properties	19
12.3 Liveness properties	19
12.4 Fairness properties	19
12.5 Precedence properties	20
13. Relationships between reasoning tasks	20
13.1 Satisfiability and validity	20
13.2 Reducing model checking to validity	20
13.3 Reducing validity to model checking	21
13.4 Complexity	22
14. LTL tableau algorithm for satisfiability	22
14.1 Relation to propositional tableau	22
15. Alpha, beta, eventuality, and literal formulae	22
15.1 Alpha formulae: conjunctive behaviour	22
15.2 Beta formulae: disjunctive behaviour	22
15.3 Eventualities	23
15.4 Literals	23
15.5 Exam flag: alpha/beta table	23
16. Closure of a formula	23
16.1 Intuition	23
16.2 Formal definition	24
17. Types	24
17.1 Intuition	24
17.2 Formal definition	24
17.3 Lemma: worlds induce types	25
17.4 Worked type example	25
17.5 Next formulae are not decomposed locally	25

18. Computing types of a set	25
18.1 Alpha rule	26
18.2 Beta rule	26
19. Pre-tableau algorithm	26
19.1 Input and initial graph	26
19.2 $\vee$ -rule	26
19.3 $\exists$ -rule	27
19.4 What the pre-tableau guarantees	28
19.5 Why the pre-tableau is not yet a model	28
20. Elimination algorithm	28
20.1 Non-type rule	28
20.2 Bad-next rule	29
20.3 Bad-future rule	29
21. Correctness observations for the tableau	29
22. Termination and satisfiability theorem	30
22.1 Termination and complexity	30
22.2 Satisfiability theorem	30
22.3 Algorithm optimality	31
23. LTL limitations	31
23.1 Only one future	31
23.2 Worlds are unstructured	31
23.3 Only future, no past	31
23.4 No clocks	31
23.5 No probabilities	32
24. LTL extensions	32
24.1 Branching-time logics	32
24.2 Temporal logics with structured worlds	32
24.3 Past modalities	32
24.4 Timed temporal logics	32
24.5 Probabilistic temporal logics	32
25. LTL and other logics	33
25.1 LTL extends propositional logic	33
25.2 LTL is orthogonal to description logics and datalog	33
25.3 Reasoning-task comparison	33
25.4 Big comparison table	33
26. Exam flags and high-value points	33
26.1 Must understand	33
26.2 Explicit “not tested / list given” flags	34
26.3 Practice flags	34
27. Connections to earlier material	34
28. Unclear transcript sections / things to re-listen to	34
Source materials used	35

Topic and scope

This lecture block introduces **Linear Temporal Logic (LTL)** as a propositional temporal logic for reasoning about infinite timelines and dynamic systems. It covers LTL syntax and semantics, transition systems, satisfiability/validity/model checking, a tableau satisfiability algorithm, and LTL's limitations/extensions/relationship to other logics.

1. Why temporal logic?

1.1 Recap: previous logics were static

The course has already covered:

- propositional logic;
- description logics;
- datalog.

For each, the course discussed syntax, semantics, reasoning tasks, entailment, algorithms, and relationships between reasoning tasks. The key limitation motivating this module is that these logics are **static**: their interpretations do not contain an inherent notion of “before”, “after”, “yesterday”, “tomorrow”, or “eventually”.

1.2 Ontological choices about time

Before choosing a temporal logic, the lecture frames time as an ontological modelling decision. Important questions include:

- Is time **continuous** or **discrete**?
- Is time **point-based** or **interval-based**?
- Is time **linear**, **branching/divergent**, or **cyclical**?
- Is time **one-directional** or **two-directional**?

The lecture's chosen logic, LTL, makes the following commitments:

linear + discrete + point-based + future-only.

So LTL models time as a single infinite sequence of time points:

$0, 1, 2, 3, \dots$

not as intervals, branching futures, or bidirectional time.

1.3 Fluid vs rigid statements

The lecture distinguishes between predicates/statements that can change over time and those that do not.

Fluid statements can change truth value over time. Examples:

Student(Momen), Child(Chris), isMarriedTo(Angeline, Brad).

Rigid statements remain fixed while the relevant objects exist. Examples:

Person(Shakespeare), isChildOf(Hamnet, Shakespeare).

The point is that knowledge representation about actions, events, workflows, programs, or changing systems needs a way to represent change over time. LTL is the course's basic temporal logic for this.

2. LTL syntax

2.1 Formal definition of LTL formulae

Let $\mathcal{P}$ be a set of propositional variables, often written:

$$\mathcal{P} = \{p, q, r, \dots\}.$$

The set of LTL formulae over $\mathcal{P}$ is the smallest set such that:

1. every propositional variable $p \in \mathcal{P}$ is an LTL formula;
2. if φ and ψ are LTL formulae, then the following are LTL formulae:

$$\neg\varphi$$

$$\varphi \vee \psi$$

$$X\varphi$$

$$\varphi U \psi.$$

Here:

- $X\varphi$ is read as “next φ ”;
- $\varphi U \psi$ is read as “ φ until ψ ”.

The core grammar deliberately uses only $\neg$, $\vee$, X , and U ; other useful operators are introduced later as syntactic sugar.

2.2 Intuition for the temporal operators

Next.

$$X\varphi$$

means that φ holds at the next time point.

Until.

$$\varphi U \psi$$

means that ψ must hold at some current-or-future time point, and φ must hold at every time point before that. The important subtlety is that ψ really must eventually occur; it is not enough for φ to hold forever.

2.3 Examples of LTL formulae

Examples given in the lecture include:

$$\neg(p \vee \neg p)$$

and

$$pU(q \vee X(\neg p \vee \neg q)).$$

The first one is just a propositional formula, but it still counts as an LTL formula. Every propositional formula is an LTL formula because LTL extends propositional logic.

2.4 Non-examples

The lecture gives examples of expressions that are not well-formed LTL formulae:

$$\neg(X \vee \neg p)$$

is not an LTL formula because X must be applied to a formula; here it is used without an argument.

$$U(q \vee X(\neg p \vee \neg q))$$

is not an LTL formula because U is binary: it needs one formula on the left and one formula on the right.

3. LTL semantics

3.1 LTL valuations

An LTL valuation is a mapping:

$$V : \mathbb{N} \rightarrow 2^{\mathcal{P}}.$$

So each natural number/time point $i \in \mathbb{N}$ is assigned a set of propositional variables:

$$V(i) \subseteq \mathcal{P}.$$

Intuitively, $V(i)$ is the set of propositional variables true at time point i . This means an LTL valuation is an **infinite timeline of propositional valuations**.

The lecture also says these time points are sometimes called **worlds**, and LTL valuations are traditionally called **Kripke structures**.

Exam flag. The lecturer explicitly said the term “Kripke structure” is not something they will test in this part. Still useful vocabulary, but not the main exam target.

3.2 Satisfaction relation

The satisfaction relation is written:

$$V, i \models \varphi.$$

Read this as:

In valuation V , at world/time point i , formula φ holds.

The semantics are defined inductively.

Atomic propositions

$$V, i \models p \quad \text{iff} \quad p \in V(i).$$

Negation

$$V, i \models \neg\varphi \quad \text{iff not} \quad V, i \models \varphi.$$

Disjunction

$$V, i \models \varphi \vee \psi \quad \text{iff} \quad V, i \models \varphi \text{ or } V, i \models \psi \text{ or both.}$$

Next

$$V, i \models X\varphi \quad \text{iff} \quad V, i + 1 \models \varphi.$$

Until

$$V, i \models \varphi U \psi$$

iff there exists some $j \geq i$ such that:

$$V, j \models \psi$$

and for every ℓ with

$$i \leq \ell < j,$$

we have:

$$V, \ell \models \varphi.$$

So ψ must occur at some j , and φ must hold continuously from i up to, but not including, j .

3.3 Common mistake: “until” does not mean “then stops”

Common mistake / high-value point. In

$$\varphi U \psi,$$

ψ must eventually hold. But φ is not required to stop holding when ψ becomes true. The semantics only require φ before the chosen ψ -point; φ may also hold at or after that point.

4. Syntactic sugar

The lecture first defines a minimal syntax, then introduces abbreviations to make formulae easier to write.

4.1 Conjunction, truth, and falsity

Conjunction is defined as:

$$\varphi \wedge \psi := \neg(\neg\varphi \vee \neg\psi).$$

Truth is defined as:

$$\top := p \vee \neg p.$$

Falsity is defined as:

$$\perp := p \wedge \neg p.$$

These are abbreviations, not primitive syntax.

4.2 Eventually and globally

The lecture uses E for “eventually”:

$$E\varphi := \top U \varphi.$$

This means φ holds at some current-or-future time point.

The lecture uses G for “globally”:

$$G\varphi := \neg E \neg \varphi.$$

This means φ holds everywhere in the future, including now. Since LTL is future-only, “globally” here means globally along the future timeline, not including any past.

5. Worked semantic examples

5.1 Basic valuation example

One valuation used in the lecture is:

$$V(0) = \{p, q, r\}$$

$$V(1) = \{p, q\}$$

$$V(2) = \{q, r\}$$

$$V(3) = \emptyset$$

$$V(4) = \{p\}$$

$$V(5) = \{p, q, r\}$$

$$V(i) = \{q\} \quad \text{for all } i \geq 6.$$

From this:

$$V, 0 \models p$$

because $p \in V(0)$.

$$V, 0 \models q$$

because $q \in V(0)$.

$$V, 1 \models \neg r$$

because $r \notin V(1)$.

$$V, 1 \models \neg r \vee p$$

because both $\neg r$ and p are true at time point 1.

$$V, 3 \models Xp$$

because p holds at time point 4.

$$V, 3 \models XXr$$

because r holds at time point 5.

5.2 Until example 1

Another valuation used in the lecture is:

$$V(0) = \{p, q\}$$

$$V(1) = \{p, q\}$$

$$V(2) = \{q, r\}$$

$$V(3) = \{q\}$$

$$V(4) = \{p\}$$

$$V(5) = \{p, q, r\}$$

$$V(i) = \{q\} \quad \text{for all } i \geq 6.$$

The lecture states:

$$V, 0 \models (p \wedge q)Ur.$$

Reason:

- choose $j = 2$;
- at $j = 2$, r holds;
- at all ℓ with $0 \leq \ell < 2$, namely $\ell = 0, 1$, $p \wedge q$ holds.

So the until condition is satisfied.

5.3 Until example 2

Using the same valuation:

$$V, 0 \models (q \vee r)U(p \wedge \neg q).$$

Reason:

- choose $j = 4$;
- at $j = 4$, p holds and q does not hold, so $p \wedge \neg q$ holds;
- for all time points $0, 1, 2, 3$, $q \vee r$ holds.

So the until formula is true at time point 0.

5.4 Until with bottom

The lecture also gives:

$$V, 2 \models \perp U(q \wedge r).$$

Reason:

- choose $j = 2$;
- $q \wedge r$ holds at time point 2;
- there are no time points ℓ with $2 \leq \ell < 2$, so the requirement that $\perp$ hold before j is vacuously satisfied.

This also illustrates the general point:

$$\perp U \varphi$$

behaves like φ when φ already holds now.

6. Useful semantic consequences

The lecture gives several consequences of the semantics.

6.1 If ψ holds now, then anything-until- ψ holds now

If

$$V, i \models \psi,$$

then for any LTL formula φ' :

$$V, i \models \varphi' U \psi.$$

Proof idea:

- pick $j = i$;
- then ψ holds at j ;
- there are no intermediate ℓ with $i \leq \ell < i$, so the “ φ' before j ” condition is vacuous.

6.2 Eventually means “at some future point”

$$V, i \models E\varphi \quad \text{iff} \quad \exists j \geq i \text{ such that } V, j \models \varphi.$$

Reason:

$$E\varphi := \top U \varphi.$$

Since $\top$ holds everywhere, the only real condition is that φ holds at some $j \geq i$.

6.3 Two eventualities do not imply one combined eventuality

The following does **not** hold in general:

$$V, i \models E\varphi \quad \text{and} \quad V, i \models E\psi \quad \Rightarrow \quad V, i \models E(\varphi \wedge \psi).$$

Reason: φ and ψ may become true at different future times. They need not ever be true together.

Lecture example:

$$V, 2 \models E\neg p$$

and

$$V, 2 \models E\neg q,$$

but

$$V, 2 \not\models E(\neg p \wedge \neg q).$$

So “eventually not p ” and “eventually not q ” do not imply “eventually neither p nor q ”.

6.4 Important unfolding property of until

The lecture highlights this as especially important:

$$V, i \models \varphi U \psi$$

implies

$$V, i \models \psi$$

or

$$V, i \models \varphi \wedge X(\varphi U \psi).$$

Intuition: if $\varphi U \psi$ holds now, then either ψ already holds now, or φ holds now and the same until formula still has to hold at the next time point.

Exam/high-value flag. The lecturer says this consequence is the most important one and will be used later in the tableau algorithm.

7. Transition systems: describing systems in LTL

7.1 Motivation

LTL was introduced not just to reason about arbitrary timelines, but to reason about **dynamic systems**, such as:

- algorithms;
- workflows;
- programs;
- traffic lights;
- autonomous-car abstractions.

The aim is to describe system properties such as:

- safety;
- invariance;
- liveness;
- fairness.

These properties are represented as LTL formulae, and the systems are represented using **transition systems**.

7.2 Formal definition of a transition system

Given a set of propositional variables $\mathcal{P}$, a transition system is:

$$\mathcal{M} = (S, \rightarrow, L, S_0)$$

where:

- S is a non-empty finite set of states;
- $\rightarrow \subseteq S \times S$ is the successor relation;
- $L : S \rightarrow 2^{\mathcal{P}}$ is a labelling function;
- $S_0 \subseteq S$ is the set of start states.

The successor relation tells us which states may follow which other states. The labelling function tells us which propositional variables are true in each state.

7.3 Traces

A trace in $\mathcal{M}$ is an infinite sequence:

$$\sigma = s_0, s_1, s_2, \dots$$

such that:

$$s_0 \in S_0,$$

$$s_i \in S,$$

and

$$s_i \rightarrow s_{i+1}$$

for every i .

Traces are infinite because these systems are intended to keep running; for example, a traffic light system should not simply stop.

7.4 Valuation associated with a trace

Given a trace

$$\sigma = s_0, s_1, s_2, \dots,$$

the associated LTL valuation V_σ is defined by:

$$V_\sigma(i) = L(s_i).$$

So a transition system generates traces, and each trace generates one LTL valuation.

7.5 Key intuition

A transition system describes **local behaviour**:

- which states may follow which states;
- what is true in each state.

Its traces describe possible behaviours of the system over time.

Its associated valuations describe only the propositional properties of those behaviours over time. State names themselves disappear from the valuation.

8. Worked transition-system example: car-driving abstraction

8.1 States and variables

The lecture uses a simple car-driving example.

States include:

EC = enter car

FSB = fasten seatbelt

D = drive

$Stop$ = stop

$Park$ = park

LC = leave car.

The propositional variables are:

S = safe

PA = paying attention.

Notation warning. S is overloaded in the lecture: it is used both as the general set of states and as the propositional variable “safe”. The state “Stop” is also abbreviated S in some slides/transcript passages. For revision, keep these separate.

8.2 Example traces

The lecture gives the following traces:

$$\sigma_1 = EC, FSB, D, Stop, D, Stop, D, Stop, \dots$$

This represents entering the car, fastening the seatbelt, then alternating forever between driving and stopping.

$$\sigma_2 = EC, LC, EC, LC, EC, LC, \dots$$

This represents repeatedly entering and leaving the car.

$$\sigma_3 = EC, FSB, D, Stop, Park, LC, EC, FSB, D, Stop, Park, LC, \dots$$

This represents a more sensible cycle: enter, fasten seatbelt, drive, stop, park, leave car, and repeat.

8.3 Associated valuations

For σ_1 , the valuation is:

$$V_{\sigma_1}(0) = \{S\}$$

$$V_{\sigma_1}(1) = \{S\}$$

$$V_{\sigma_1}(2) = \{PA\}$$

$$V_{\sigma_1}(3) = \{S, PA\}$$

$$V_{\sigma_1}(4) = \{PA\}$$

$$V_{\sigma_1}(5) = \{S, PA\}$$

and then it alternates between $\{PA\}$ and $\{S, PA\}$.

For σ_2 :

$$V_{\sigma_2}(0) = \{S\}$$

$$V_{\sigma_2}(1) = \{S, PA\}$$

$$V_{\sigma_2}(2) = \{S\}$$

$$V_{\sigma_2}(3) = \{S, PA\}$$

and so on.

For σ_3 :

$$V_{\sigma_3}(0) = \{S\}$$

$$V_{\sigma_3}(1) = \{S\}$$

$$V_{\sigma_3}(2) = \{PA\}$$

$$V_{\sigma_3}(3) = \{S, PA\}$$

$$V_{\sigma_3}(4) = \{S, PA\}$$

$$V_{\sigma_3}(5) = \{S, PA\}$$

$$V_{\sigma_3}(6) = \{S\}$$

$$V_{\sigma_3}(7) = \{S\}.$$

8.4 Important abstraction point

The valuation forgets why a proposition is true.

For example, $\{S, PA\}$ may occur because the system is in the “stop” state, or because it is in the “leave car” state. In the LTL valuation, both look the same: only S and PA are recorded.

9. LTL reasoning tasks

9.1 Satisfaction at a time point

Given an LTL formula φ and valuation V :

V satisfies φ at i

iff

$$V, i \models \varphi.$$

This is the most local notion: truth at one specific time point.

9.2 Satisfaction by a valuation

V satisfies φ

iff

$$V, 0 \models \varphi.$$

So satisfaction by a valuation means truth at the initial time point.

9.3 Satisfiable in a valuation

φ is satisfiable in V

iff there exists some i such that:

$$V, i \models \varphi.$$

So the formula has to hold somewhere in that valuation.

9.4 Valid in a valuation

φ is valid in V

iff for every i :

$$V, i \models \varphi.$$

So the formula holds everywhere along that one valuation.

9.5 Satisfiable

φ is satisfiable

iff there exists some valuation V such that:

$$V, 0 \models \varphi.$$

9.6 Valid

φ is valid

iff every valuation V satisfies φ , i.e.

$$V, 0 \models \varphi$$

for every V .

Exam/high-value distinction. LTL validity here means truth at time point 0 in every valuation, not truth at every time point in every valuation. “Valid in V ” is the global-in-one-valuation notion.

10. Worked reasoning-task examples

Using V_{σ_1} from the car example:

10.1 Basic satisfaction

$$V_{\sigma_1}, 1 \models S.$$

At time point 1, the valuation contains S .

Also:

$$V_{\sigma_1} \models S$$

because S holds at time point 0.

10.2 Until property

$$V_{\sigma_1} \models SUPA.$$

At the beginning, S holds until PA holds. In the given trace, PA holds at time point 2, and S holds before that.

10.3 Global until property

For each time point i :

$$V_{\sigma_1}, i \models SUPA.$$

Therefore:

$$V_{\sigma_1} \models G(SUPA).$$

Equivalently, $SUPA$ is valid in V_{σ_1} .

10.4 Formula that is not valid in the valuation

The formula

$$(X \neg S) \Rightarrow PA$$

is not valid in V_{σ_1} .

Reason: at time point 1,

$$X \neg S$$

holds because at time point 2, S is false. But at time point 1, PA is also false. So the implication fails at time point 1.

11. Model checking

11.1 Formal definition

Let φ be an LTL formula and let

$$\mathcal{M} = (S, \rightarrow, L, S_0)$$

be a transition system.

We say φ holds in $\mathcal{M}$, written:

$$\mathcal{M} \models \varphi,$$

iff for every trace σ in $\mathcal{M}$, and for the associated valuation V_σ , we have:

$$V_\sigma, 0 \models \varphi.$$

So model checking asks whether every possible run of the system satisfies the formula at the start.

11.2 Universal vs existential model checking

The lecture's definition is **universal model checking**:

all traces must satisfy φ .

There is also an existential version, where some trace must satisfy φ . The lecture notes that the existential version is dual to the universal version:

$$\mathcal{M} \not\models \neg \varphi$$

iff there exists a trace in $\mathcal{M}$ whose valuation satisfies φ .

Exam/high-value point. The course definition of model checking is universal unless stated otherwise.

11.3 Model-checking examples from the car system

The lecture gives:

$$\mathcal{M} \models S.$$

Every trace starts in the enter-car state, where S holds.

$$\mathcal{M} \models SUPA.$$

Every trace begins with S holding until PA holds.

$$\mathcal{M} \models G(SUPA).$$

At every point in every trace, $SUPA$ holds.

But:

$$\mathcal{M} \not\models G((X \neg S) \Rightarrow PA).$$

This fails because the system has a trace/time point where the next state is unsafe, but the current state is not already paying attention.

12. Types of system properties

The lecture briefly classifies system properties expressible or checked using LTL.

12.1 Local properties

Local properties concern what happens between a state and its next state.

Example:

$$\mathcal{M} \models \neg S \Rightarrow XS.$$

If the system is not safe now, then it is safe at the next time point.

12.2 Safety properties

Safety properties express that “bad things do not happen”.

Example:

$$\mathcal{M} \models G(\neg PA \Rightarrow S).$$

If the system is not paying attention, then it is safe.

12.3 Liveness properties

Liveness properties express that “good things eventually happen”.

The lecture’s car transition system does not satisfy some natural liveness expectations, because it allows infinite traces that keep cycling between driving and stopping forever without parking or leaving the car.

Examples from the slides include:

$$\mathcal{M} \not\models G(S \Rightarrow E \neg S)$$

and

$$\mathcal{M} \not\models G(\neg S \Rightarrow ES).$$

The broader lesson is that transition systems can contain unrealistic infinite traces unless the model rules them out.

12.4 Fairness properties

Fairness properties are often similar to liveness properties. Example intuition: every request is eventually replied to. The lecture mentions fairness for completeness but does not develop it in detail.

12.5 Precedence properties

Precedence properties say that one event must occur before another, such as “look left and right before crossing the road”.

The lecture notes that these often require looking backwards in time, which plain future-only LTL does not directly support.

13. Relationships between reasoning tasks

13.1 Satisfiability and validity

As in propositional logic:

φ is satisfiable

iff

$\neg\varphi$ is not valid.

So a satisfiability tester can be used to build a validity tester, and vice versa.

13.2 Reducing model checking to validity

Given a transition system:

$$\mathcal{M} = (S, \rightarrow, L, s_0),$$

the lecture constructs a formula $\varphi_{\mathcal{M}}$ that represents the transition system. It introduces a new propositional variable p_s for each state $s \in S$.

The formula has four conceptual parts.

Initial state

$$p_{s_0}$$

This says the system starts in the start state.

Exactly one state at each time point

$$G \left(\bigvee_{s \in S} \left(p_s \wedge \bigwedge_{s' \neq s} \neg p_{s'} \right) \right).$$

This says that at every time point, exactly one state variable is true.

Successor relation is respected

$$G \left(\bigwedge_{s \in S} \left(p_s \Rightarrow X \left(\bigvee_{s \rightarrow s'} p_{s'} \right) \right) \right).$$

This says that if the system is in state s , then at the next time point it must be in one of s 's successor states.

State labelling is respected

$$G \left(\bigwedge_{s \in S} \left(p_s \Rightarrow \left(\bigwedge_{p \in L(s)} p \wedge \bigwedge_{p \notin L(s)} \neg p \right) \right) \right).$$

This says that if the system is in state s , then exactly the propositions in $L(s)$ hold.

Then:

$$\mathcal{M} \models \psi$$

iff

$$\varphi_{\mathcal{M}} \Rightarrow \psi$$

is valid.

The lecture states this is a polynomial reduction.

13.3 Reducing validity to model checking

Given a formula ψ with propositional variables $\mathcal{P}_\psi$, construct a “universal” transition system:

$$\mathcal{M}_\psi = (S, \rightarrow, L, S)$$

where:

$$S := 2^{\mathcal{P}_\psi}$$

so every subset of the propositional variables is a state;

$$\rightarrow := S \times S$$

so every state can transition to every state;

$$L(s) := s.$$

Then:

$$\psi \text{ is valid}$$

iff

$$\mathcal{M}_\psi \models \psi.$$

This reduction is not polynomial: the number of states is exponential in the number of propositional variables in ψ .

13.4 Complexity

The lecture says:

- model checking can be done in time linear in the number of transition-system states and exponential in the size of the formula;
- transition systems are often huge, but formulae are often small;
- dedicated model checkers work very well in practice;
- satisfiability, validity, and model checking all have worst-case complexity PSPACE-complete.

Exam flag. The lecturer explicitly says PSPACE-completeness itself will not be tested in detail, but it is useful context. It is harder than the NP-complete propositional logic problems seen earlier.

14. LTL tableau algorithm for satisfiability

14.1 Relation to propositional tableau

The lecture reminds students of the propositional tableau algorithm:

- input in negation normal form;
- works on sets of sets of formulae;
- $\wedge$ -rule adds both conjuncts;
- $\vee$ -rule branches into cases, adding one disjunct to each copy.

For LTL, the tableau is more complicated because:

- we do **not** use negation normal form;
- we must reason by cases for more formula types than just disjunction;
- we must account for different time points in an infinite timeline.

Exam flag. The lecturer says that if the propositional tableau is not remembered, students should go back to Weeks 2-3. That is a prerequisite idea for understanding the LTL tableau.

15. Alpha, beta, eventuality, and literal formulae

15.1 Alpha formulae: conjunctive behaviour

Alpha formulae behave like conjunctions: satisfying the whole formula forces satisfaction of all components.

Alpha formula	Components
$\neg\neg\alpha$	α
$\alpha_1 \wedge \alpha_2$	α_1, α_2
$\neg(\alpha_1 \vee \alpha_2)$	$\neg\alpha_1, \neg\alpha_2$
$\neg X\alpha$	$X\neg\alpha$
$\neg(\varphi U \psi)$	$\neg\psi, \neg\varphi \vee \neg X(\varphi U \psi)$

For example, if $\neg(\varphi U \psi)$ holds, then ψ cannot hold now, and either φ fails now or the until formula fails at the next point.

15.2 Beta formulae: disjunctive behaviour

Beta formulae behave like disjunctions: satisfying the whole formula requires at least one component.

Beta formula	Components
$\alpha_1 \vee \alpha_2$	α_1, α_2
$\neg(\alpha_1 \wedge \alpha_2)$	$\neg\alpha_1, \neg\alpha_2$
$\varphi U \psi$	$\psi, \varphi \wedge X(\varphi U \psi)$

The key beta case is:

$$\varphi U \psi.$$

It can be satisfied either because ψ holds now, or because φ holds now and $\varphi U \psi$ remains true at the next time point.

15.3 Eventualities

Eventualities are formulae of the form:

$$\varphi U \psi.$$

They impose an eventual satisfaction requirement: ψ must eventually happen. This matters later for the bad-future elimination rule.

15.4 Literals

Literals are:

$$p$$

and

$$\neg p.$$

15.5 Exam flag: alpha/beta table

Exam flag. The lecturer explicitly says students will not be asked to memorise the full list of alpha and beta formulae. If the list is needed in an exam-style task, it will be given. The important thing is to understand how the components behave.

16. Closure of a formula

16.1 Intuition

The closure of a formula contains all formulae relevant for reasoning about it:

- the formula itself;
- its subformulae;
- single negations;
- components of alpha and beta formulae.

It is finite and its size is linear in the length of the original formula.

16.2 Formal definition

Let ϕ be an LTL formula. Its closure $cl(\phi)$ is the smallest set satisfying:

1.

$$\top, \phi \in cl(\phi).$$

2. All subformulae of ϕ are in $cl(\phi)$.

3. If $\phi_1 \in cl(\phi)$ and ϕ_1 is neither of the form $\neg\phi_2$ nor of the form $X\neg\phi_2$ for some $\neg X\phi_2 \in cl(\phi)$, then:

$$\neg\phi_1 \in cl(\phi).$$

4. If $\phi_1 \in cl(\phi)$ and ϕ_1 is of kind α or β , then its components are in $cl(\phi)$.

[UNCLEAR] The transcript and slide notation around the special $X\neg\phi_2$ exception is dense. The operational point is clear: the closure is closed under relevant **single** negations without generating infinite chains like $p, \neg p, \neg\neg p, \dots$

17. Types

17.1 Intuition

A type is a maximal, locally consistent set of formulae from a closure. Intuitively, a type describes what could hold at one world/time point.

A type:

- obeys alpha formulae;
- obeys beta formulae;
- is clash-free.

17.2 Formal definition

Let ϕ be an LTL formula and $cl(\phi)$ its closure.

A set

$$\Gamma \subseteq cl(\phi)$$

is a ϕ -type iff:

1. For every alpha formula ϕ' :

$$\phi' \in \Gamma$$

iff both its components are in Γ .

2. For every beta formula ϕ' :

$$\phi' \in \Gamma$$

iff at least one of its components is in Γ .

3. Γ is clash-free: it does not contain both some formula ϕ' and its negation $\neg\phi'$.

17.3 Lemma: worlds induce types

If ψ is an LTL formula, V is a valuation, and

$$V, i \models \psi,$$

then define:

$$t(i) := \{\varphi \in cl(\psi) \mid V, i \models \varphi\}.$$

The lemma says $t(i)$ is a type.

Intuition: all the formulae from the closure that actually hold at a real world must be locally consistent and must respect alpha/beta decomposition.

17.4 Worked type example

The lecture gives a valuation where:

$$V, 0 \models (p \vee q)U(p \wedge q \wedge r).$$

Then the type at 0 includes:

$$p, \quad q, \quad r, \quad p \vee q, \quad p \wedge q \wedge r, \quad (p \vee q)U(p \wedge q \wedge r).$$

This is a type because the until formula is beta, and one of its components, namely $p \wedge q \wedge r$, holds now.

17.5 Next formulae are not decomposed locally

The lecture stresses that:

$$X\psi$$

is neither alpha nor beta and has no local components. If $X\psi$ holds at time i , ψ is required at time $i + 1$, not at time i . Therefore ψ is not automatically included in the current type.

18. Computing types of a set

Given a set M of LTL formulae:

$$cl(M) := \bigcup_{\varphi \in M} cl(\varphi).$$

The types of M are:

$$ts(M) := \{t \subseteq cl(M) \mid t \text{ is a type and } M \subseteq t\}.$$

To compute $ts(M)$:

1. start with M ;
2. exhaustively apply the alpha rule;
3. exhaustively apply the beta rule;
4. drop sets that contain a clash.

18.1 Alpha rule

If a set S contains an alpha formula φ but not both of its components, add both components to S . This generalises the propositional $\wedge$ -rule.

18.2 Beta rule

If a set S contains a beta formula φ but none of its components, replace S with two copies:

$$S \cup \{\varphi_1\}$$

and

$$S \cup \{\varphi_2\},$$

where φ_1, φ_2 are the beta components.

This generalises the propositional $\vee$ -rule.

19. Pre-tableau algorithm

19.1 Input and initial graph

Input: an LTL formula ψ .

Initialise an edge-labelled graph:

$$G = (\{e_\psi, e_\top\}, E_\vee \cup E_X, L)$$

with:

$$E_\vee = E_X = \emptyset,$$

$$L(e_\psi) = \{\psi\},$$

$$L(e_\top) = \{\top\}.$$

There are two kinds of edges:

- $E_\vee$ -edges, used to expand local case distinctions;
- E_X -edges, used to move to next-time-point requirements.

19.2 $\vee$ -rule

If there is a node $e \in V$ such that:

- $L(e)$ is not a type;
- e has no $E_\vee$ -successor,

then compute:

$$T_e := ts(L(e)).$$

For each $t \in T_e$:

- if there is already a node e' with $L(e') = t$, add:

$$(e, e') \in E_V;$$

- otherwise create a new node e' , set:

$$L(e') := t,$$

and add:

$$(e, e') \in E_V.$$

This rule ensures all local cases are explored.

19.3 X -rule

If there is a node $e \in V$ such that:

- $L(e)$ is a type;
- e has no E_X -successor,

then compute:

$$X_e := \{\varphi \mid X\varphi \in L(e)\}.$$

If

$$X_e = \emptyset,$$

add:

$$(e, e_\top) \in E_X.$$

If there is already a node e' with:

$$L(e') = X_e,$$

add:

$$(e, e') \in E_X.$$

Otherwise create a new node e' , set:

$$L(e') := X_e,$$

and add:

$$(e, e') \in E_X.$$

This rule ensures all next-time requirements are explored.

19.4 What the pre-tableau guarantees

After exhaustive application:

- if $(e, e') \in E_V$, then:

$$L(e) \subseteq L(e');$$

- each type node has an E_X -successor.

19.5 Why the pre-tableau is not yet a model

Things can still go wrong:

1. A node may have no possible type extension.

Example:

$$ts(\{p \wedge \neg p\}) = \emptyset.$$

2. An eventuality may never be fulfilled.

Example:

$$pU(p \wedge \neg p)$$

cannot be satisfied because $p \wedge \neg p$ can never hold.

20. Elimination algorithm

The elimination algorithm takes a pre-tableau:

$$G = (V, E_V \cup E_X, L)$$

and repeatedly removes bad nodes.

20.1 Non-type rule

If there is a node $e \in V$ with $L(e)$ not a type, then for each $e_1, e_2 \in V$ such that:

$$(e_1, e) \in E_X$$

and

$$(e, e_2) \in E_V,$$

add:

$$(e_1, e_2) \in E_X.$$

Then remove e from V and remove its incident edges.

Intuition: remove intermediate non-type nodes by connecting their incoming X -predecessors directly to their type alternatives.

20.2 Bad-next rule

If there is a node $e \in V$ with no E_X -successor, remove e .

Reason: in LTL, every time point must have a next time point. A node that cannot continue cannot represent a point on an infinite timeline.

Example problem:

$$X(p \wedge \neg p) \in L(e).$$

The next state would have to satisfy a contradiction, so the path fails.

20.3 Bad-future rule

If there is a node $e \in V$ for which there is no lasso satisfying all eventualities in $L(e)$, remove e .

A **lasso** is an eventually cyclic X -path starting from e . It represents an infinite path using a finite graph: move along a finite stem, then loop forever.

A lasso satisfies all eventualities in $L(e)$ iff for each eventuality:

$$\varphi_1 U \varphi_2 \in L(e),$$

there is some node e' on the lasso such that:

$$\varphi_2 \in L(e').$$

Intuition: the bad-future rule removes nodes where an until formula is postponed forever and its right-hand side never occurs.

21. Correctness observations for the tableau

Let G be the result after applying the elimination algorithm, and let e be a node in G .

The lecture lists these claims:

1. $L(e)$ is a type.
2. e has an outgoing X -edge.
3. If

$$X\varphi \in L(e),$$

then

$$\varphi \in L(e')$$

for each X -successor e' of e .

4. If

$$\varphi U \psi \in L(e),$$

then

$$\psi \in L(e')$$

for some node e' reachable from e via an X -path.

5. For nodes $e_1, e_2, \dots$ on an X -path from e , if

$$\varphi U \psi \in L(e),$$

and i is the minimal index such that:

$$\psi \in L(e_i),$$

then for all $j < i$:

$$\varphi, \varphi U \psi \in L(e_j).$$

This matches the semantics of until: before ψ is reached, φ must keep holding, and the until obligation remains active.

[UNCLEAR] The transcript mentions a typo around this fifth claim, and the slide wording also appears to end with $L(e)$ where $L(e_j)$ is intended. The corrected version above is the semantically coherent one.

22. Termination and satisfiability theorem

22.1 Termination and complexity

The lecture states:

- the pre-tableau algorithm terminates;
- the elimination algorithm terminates;
- both can be carried out in time exponential in $|\psi|$.

Reason:

- $cl(\psi)$ is linear in $|\psi|$;
- node labels are unique subsets of $cl(\psi)$;
- there are exponentially many such subsets;
- the algorithms monotonically build up and then remove from a finite graph.

22.2 Satisfiability theorem

Let G_1 be the pre-tableau constructed for ψ , and let G_2 be the result of applying the elimination algorithm.

Then:

ψ is satisfiable

iff there is a node $e \in G_2$ such that:

$$\psi \in L(e).$$

The proof idea goes both ways:

- if ψ is satisfiable, a real valuation satisfying ψ can be used to “watch” suitable nodes survive the algorithm;
- if a surviving graph contains ψ , one can construct a valuation satisfying ψ .

22.3 Algorithm optimality

The lecturer says the presented tableau algorithm is understandable but not optimal in space. LTL satisfiability is PSPACE-complete, but this tableau presentation uses exponential space. Other algorithms can use only polynomial space.

23. LTL limitations

23.1 Only one future

LTL has a single linear future. It cannot distinguish:

in one possible tomorrow, I will have two breakfasts

from

in all possible tomorrows, I will have two breakfasts.

So plain LTL cannot express branching possibility versus necessity over different possible futures.

23.2 Worlds are unstructured

Each world is propositional. LTL cannot directly express internal structure such as individuals and relations.

Example the lecture says LTL cannot express directly:

Tomorrow I will be late, but I will know somebody who is not late.

This requires objects and a binary relation such as $knows(x, y)$, which plain propositional LTL lacks.

23.3 Only future, no past

Plain LTL cannot talk about the past.

Example not expressible directly:

If it rained yesterday, I will take an umbrella tomorrow.

LTL can talk about tomorrow, but not yesterday.

23.4 No clocks

LTL time points are equidistant and abstract. There are no clocks or time measurements.

So plain LTL cannot naturally express:

something happens within five minutes

or

at most ten seconds pass before an event.

23.5 No probabilities

Everything is either true or false. Plain LTL cannot say:

“tomorrow I will have two breakfasts with probability 83%”.

This limitation applies both to LTL formulae and to transition systems: transitions either exist or do not exist; they do not carry probabilities.

24. LTL extensions

24.1 Branching-time logics

To model several possible futures, people use computational tree logics such as:

- CTL;
- CTL*.

These allow the timeline to branch like a tree, making it possible to distinguish properties that hold in some futures from properties that hold in all futures.

24.2 Temporal logics with structured worlds

LTL can be combined with first-order logic or description logics so that each world contains a structured interpretation. This allows relations between individuals, such as people knowing other people, to be represented.

24.3 Past modalities

LTL can be extended with past operators such as:

- yesterday;
- since;
- globally in the past.

The lecture states that LTL with past modalities is equally expressive as LTL without past modalities, but past modalities make formulae more readable and writable.

24.4 Timed temporal logics

Timed extensions add clocks and comparisons, such as:

$\geq$, $<$, ...

These allow statements about how much time passes before or after an event.

24.5 Probabilistic temporal logics

Probabilistic extensions use probabilistic transition systems, described in the lecture as Markov processes, and formula operators such as:

$P_{\geq 80\%}\varphi$.

This allows formulae to express probability thresholds.

25. LTL and other logics

25.1 LTL extends propositional logic

An LTL formula without temporal operators such as X , U , E , or G is just a propositional formula.

Semantically, LTL has a propositional valuation at each time point, so it can be seen as an infinite timeline of propositional valuations.

25.2 LTL is orthogonal to description logics and datalog

LTL has:

- time points/worlds;
- built-in temporal relations such as before/after;
- no user-defined relations like *hasDaughter*, *hasPart*, *likes*, or *knows*.

Description logics and datalog have structured interpretations with individuals and relations, but do not inherently have LTL's infinite timeline.

25.3 Reasoning-task comparison

LTL has reasoning tasks similar to earlier logics:

- satisfiability;
- validity.

But they become more complex because of time, local/global distinctions, and infinite timelines.

LTL also adds model checking as a central reasoning task.

25.4 Big comparison table

The slide deck's final comparison table says the LTL column has:

- unary-only predicates, like propositional logic;
- conjunction, disjunction, and negation;
- semantics as a valuation at each point on an infinite timeline;
- main reasoning tasks: satisfiabilities, validities, model checking;
- complexity: PSPACE-complete;
- algorithm considered: LTL tableau, no normal form;
- KR use: testing whether systems satisfy liveness/safety/etc. conditions.

26. Exam flags and high-value points

26.1 Must understand

- The semantics of X and U , especially the exact quantifiers in the definition of until.
- The difference between:
 - satisfaction at a time point;
 - satisfaction by a valuation;
 - satisfiable in a valuation;
 - valid in a valuation;
 - satisfiable;
 - valid.
- Model checking is universal in this course: all traces must satisfy the formula.
- $\varphi U \psi$ is an eventuality: ψ must eventually occur.

- The unfolding intuition:

$$\varphi U \psi \Rightarrow \psi \vee (\varphi \wedge X(\varphi U \psi)).$$

- Transition systems generate traces; traces generate valuations.
- The valuation forgets state names and keeps only propositional labels.
- The tableau’s bad-future rule is needed because an until formula can be postponed forever in a cyclic graph.
- Model checking is often practically easier than satisfiability because it is linear in the transition-system size and exponential in the formula size.

26.2 Explicit “not tested / list given” flags

- Full alpha/beta formula list: lecturer said they will not ask students to memorise it; if needed, it will be provided.
- “Kripke structure” terminology: lecturer said this will not be tested.
- PSPACE-completeness details: lecturer said this will not be tested in detail, though it is important context.

26.3 Practice flags

- Revisit propositional tableau from Weeks 2-3 if rusty.
- Work through the LTL tableau on a very small formula by hand.
- Be able to explain why a bad lasso violates an eventuality.
- Be able to construct or inspect a trace of a transition system and derive its valuation.

27. Connections to earlier material

- **Propositional logic:** LTL extends propositional logic syntactically and semantically.
- **Propositional tableau:** LTL tableau generalises the $\wedge$ - and $\vee$ -rules using alpha and beta formulae.
- **Description logics / datalog:** LTL is orthogonal: it has time but no structured individuals/relations.
- **Finite-state automata:** transition systems are close to finite-state automata, except that states are labelled rather than transitions.
- **KR ontological commitments:** choosing LTL commits to linear, discrete, point-based, future-only time.
- **System verification:** LTL is used to test safety, liveness, fairness, and related properties of dynamic systems.

28. Unclear transcript sections / things to re-listen to

- **[UNCLEAR]** The course number is transcribed inconsistently as “come 641”, “6441”, and “64401”. The slides identify the course as **COMP64401 Logics for Knowledge Representation and Reasoning**.
- **[UNCLEAR]** The transcript phrase “has doTERRA has power to like snows” is garbled. The slide/table context indicates examples like *hasDaughter*, *hasPart*, *likes*, and probably *knows*.
- **[UNCLEAR]** The closure definition’s exception involving $X\neg\phi_2$ and $\neg X\phi_2 \in cl(\phi)$ is dense in both transcript and slide. Re-listen around the closure definition if you need the exact formal clause for coursework.
- **[UNCLEAR]** The tableau correctness claim 5 appears to contain a typo in the transcript/slide: it should refer to labels $L(e_j)$ for earlier nodes on the path, not always $L(e)$.

- **[UNCLEAR]** The hand-drawn transition-system labels are hard to read in the slide image, but the following valuation slides clarify the labels used in the examples.
- **[UNCLEAR]** The final table appears to say “liveness/safety”; this should be read as **live-ness/safety**.

Source materials used

- 5.1LTLSyntaxSemantics-English.txt
- 5.3LTLLTransitionSystems-English.txt
- 5.4LTLLReasoningTasks-English.txt
- 5.5LTLLTableau-English.txt
- 5.6LTLLimitations-English.txt
- Week9+10 (1).pdf